

Sprint Documentation

Sprint 13	Start Date: 17/03/2026	Final Date: 23/03/2026
Projetc Name:	Civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Carlos Alberto Ferreira Candido		
Giovanni da Silva Nascimento		

Sprint Backlog

Task #	Description	Start Date	Final date
#59	Validação dos campos da secretaria	17/03/2026	23/03/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#59	Garantir que o cadastro de secretarias possua validações completas de integridade, padronização e consistência dos dados, evitando informações inválidas ou duplicadas no sistema.	Carlos Alberto Ferreira Candido, Giovanni da Silva Nascimento	Completo	20 horas	20 horas

Sprint Description

🚩 Tarefa: Validações de Cadastro de Secretaria

🎯 Objetivo

Garantir que o cadastro de secretarias possua validações completas de integridade, padronização e consistência dos dados, evitando informações inválidas ou duplicadas no sistema.

📋 Descrição

Implementar validações no backend para os campos do cadastro de secretaria, assegurando:

- Dados obrigatórios

- Formatação correta
 - Padronização (normalização)
 - Unicidade no banco
-

☑ Critérios de Aceite

◆ Campos Obrigatórios

- - Validar que os campos obrigatórios não estão vazios:
 - nome
 - descricao
 - cnpj
 - nomeRazaoSocial
 - email
 - telefone
 - logradouro
 - numero
 - bairro
 - cidade
 - estado
 - cep
 - situacao
-

◆ Normalização de Dados

- Permitir entrada de cnpj, cep e telefone com máscara
- Remover caracteres não numéricos antes de processar:
 - CNPJ
 - CEP
 - Telefone

Armazenar apenas números no banco (quando aplicável)

◆ Validação de CNPJ

- CNPJ deve conter 14 dígitos após normalização

CNPJ deve ser válido

CNPJ não pode estar duplicado no banco

Rejeitar CNPJs inválidos (ex: todos números iguais)

◆ Validação de Email

- Validar formato de email válido

Limitar tamanho (máx. 255 caracteres)

Validar unicidade no sistema

◆ Validação de Nome

- - nome:
 - Mínimo de 3 caracteres
 - Máximo de 150 caracteres
-

◆ Validação de Razão Social

- - nomeRazaoSocial:
 - Mínimo de 3 caracteres
 - Máximo de 200 caracteres
-

◆ Validação de Descrição

- - descricao:
 - Não pode ser vazia
 - Definir tamanho máximo (ex: 500 caracteres)
-

◆ Validação de Telefone

- Deve conter apenas números após normalização

Deve possuir entre 10 e 11 dígitos (com DDD)

◆ Validação de Endereço

- logradouro: obrigatório e com tamanho máximo (max: 200)

numero: obrigatório

bairro: obrigatório

cidade: obrigatório

- estado: obrigatório
 - Deve conter uma UF válida (ex: SP, RJ, MG)

- cep:
 - Deve conter 8 dígitos após normalização
-

◆ Validação de Situação

- - Aceitar apenas valores válidos:
 - 1 = Ativo
 - 2 = Inativo
-

◆ Validação de idSecretaria

- Deve ser ignorado no cadastro (gerado automaticamente) ou validado conforme regra do sistema

Não permitir valores negativos ou inválidos

Observações e Sugestões

- Podem utilizar DataAnnotations para validações básicas, mas não se prendam a isso. Utilize validações mais robustas para regras de negócio.
- Criar métodos reutilizáveis para normalização (CNPJ, CEP, telefone).
- Validar unicidade diretamente no banco (principalmente CNPJ).
- Validações preferencialmente na camada SERVICE.
- Utilizar enums para situacao.
- Documentar as alterações/adições.
- Verifiquem o funcionamento antes do envio.

Sprint Results

Foi adicionado o método sanitize que remove todos os caracteres especiais dos campos.

```

private static string Sanitize(string? valor, bool digitsOnly = false)
{
    if (string.IsNullOrEmpty(valor))
    {
        return string.Empty;
    }

    var sanitized = valor.Trim();
    return digitsOnly
        ? Regex.Replace(sanitized, "[^0-9]", string.Empty)
        : sanitized;
}

private static void NormalizarCampos(SecretariaDTO secretariaDTO)
{
    secretariaDTO.Descricao = Sanitize(secretariaDTO.Descricao);
    secretariaDTO.Nome = Sanitize(secretariaDTO.Nome);
    secretariaDTO.Logradouro = Sanitize(secretariaDTO.Logradouro);
    secretariaDTO.Numero = Sanitize(secretariaDTO.Numero);
    secretariaDTO.Bairro = Sanitize(secretariaDTO.Bairro);
    secretariaDTO.NomeRazaoSocial = Sanitize(secretariaDTO.NomeRazaoSocial);
    secretariaDTO.Email = Sanitize(secretariaDTO.Email).ToLowerInvariant();
    secretariaDTO.Cidade = Sanitize(secretariaDTO.Cidade);
    secretariaDTO.Estado = Sanitize(secretariaDTO.Estado).ToUpperInvariant();

    secretariaDTO.Cnpj = Sanitize(secretariaDTO.Cnpj, digitsOnly: true);
    secretariaDTO.Cep = Sanitize(secretariaDTO.Cep, digitsOnly: true);
    secretariaDTO.Telefone = Sanitize(secretariaDTO.Telefone, digitsOnly: true);
}

```

Foi adicionado também um método para validar se os campos estão vazios, se atendem o requisito mínimo de caracteres e se existe apenas um no banco com aquele mesmo campo.

```

public async Task ValidarCadastroAsync(SecretariaDTO entityDTO, int? id = null)
{
    if (entityDTO is null)
    {
        throw new ArgumentException("Dados da secretaria são obrigatórios.");
    }

    if (id.HasValue && id.Value <= 0)
    {
        throw new ArgumentException("Id da secretaria inválido.");
    }

    if (id.HasValue)
    {
        var existingEntity = await _secretariaRepository.GetById(id.Value);
        if (existingEntity is null)
        {
            throw new KeyNotFoundException($"Entidade com id {id.Value} não encontrada.");
        }
    }

    entityDTO.IdSecretaria = id ?? 0;

    NormalizarCampos(entityDTO);
    ValidarCampos(entityDTO);
    await ValidarUnicidade(entityDTO, id);
}

```


1 referência

```
private async Task ValidarUnicidade(SecretariaDTO secretariaDTO, int? ignoreId)
{
    var cnpjDuplicado = await _secretariaRepository.ExistsByCnpjAsync(secretariaDTO.Cnpj, ignoreId);
    if (cnpjDuplicado)
    {
        throw new ArgumentException("CNPJ já cadastrado no sistema.");
    }

    var emailDuplicado = await _secretariaRepository.ExistsByEmailAsync(secretariaDTO.Email, ignoreId);
    if (emailDuplicado)
    {
        throw new ArgumentException("E-mail já cadastrado no sistema.");
    }
}
```

1 referência

```
private static void ValidarCampos(SecretariaDTO secretariaDTO)
{
    ValidarObrigatorio(secretariaDTO.Nome, nameof(secretariaDTO.Nome));
    ValidarObrigatorio(secretariaDTO.Descricao, nameof(secretariaDTO.Descricao));
    ValidarObrigatorio(secretariaDTO.Cnpj, nameof(secretariaDTO.Cnpj));
    ValidarObrigatorio(secretariaDTO.NomeRazaoSocial, nameof(secretariaDTO.NomeRazaoSocial));
    ValidarObrigatorio(secretariaDTO.Email, nameof(secretariaDTO.Email));
    ValidarObrigatorio(secretariaDTO.Telefone, nameof(secretariaDTO.Telefone));
    ValidarObrigatorio(secretariaDTO.Logradouro, nameof(secretariaDTO.Logradouro));
    ValidarObrigatorio(secretariaDTO.Numero, nameof(secretariaDTO.Numero));
    ValidarObrigatorio(secretariaDTO.Bairro, nameof(secretariaDTO.Bairro));
    ValidarObrigatorio(secretariaDTO.Cidade, nameof(secretariaDTO.Cidade));
    ValidarObrigatorio(secretariaDTO.Estado, nameof(secretariaDTO.Estado));
    ValidarObrigatorio(secretariaDTO.Cep, nameof(secretariaDTO.Cep));

    ValidarTamanho(secretariaDTO.Nome, 3, 150, "Nome");
    ValidarTamanho(secretariaDTO.NomeRazaoSocial, 3, 200, "NomeRazaoSocial");
    ValidarTamanho(secretariaDTO.Descricao, 1, 500, "Descricao");
    ValidarTamanho(secretariaDTO.Logradouro, 1, 200, "Logradouro");

    if (secretariaDTO.IdSecretaria < 0)
    {
        throw new ArgumentException("IdSecretaria não pode ser negativo.");
    }
}
```

1 referência

```
private static void ValidarCampos(SecretariaDTO secretariaDTO)
{
    ValidarObrigatorio(secretariaDTO.Nome, nameof(secretariaDTO.Nome));
    ValidarObrigatorio(secretariaDTO.Descricao, nameof(secretariaDTO.Descricao));
    ValidarObrigatorio(secretariaDTO.Cnpj, nameof(secretariaDTO.Cnpj));
    ValidarObrigatorio(secretariaDTO.NomeRazaoSocial, nameof(secretariaDTO.NomeRazaoSocial));
    ValidarObrigatorio(secretariaDTO.Email, nameof(secretariaDTO.Email));
    ValidarObrigatorio(secretariaDTO.Telefone, nameof(secretariaDTO.Telefone));
    ValidarObrigatorio(secretariaDTO.Logradouro, nameof(secretariaDTO.Logradouro));
    ValidarObrigatorio(secretariaDTO.Numero, nameof(secretariaDTO.Numero));
    ValidarObrigatorio(secretariaDTO.Bairro, nameof(secretariaDTO.Bairro));
    ValidarObrigatorio(secretariaDTO.Cidade, nameof(secretariaDTO.Cidade));
    ValidarObrigatorio(secretariaDTO.Estado, nameof(secretariaDTO.Estado));
    ValidarObrigatorio(secretariaDTO.Cep, nameof(secretariaDTO.Cep));

    ValidarTamanho(secretariaDTO.Nome, 3, 150, "Nome");
    ValidarTamanho(secretariaDTO.NomeRazaoSocial, 3, 200, "NomeRazaoSocial");
    ValidarTamanho(secretariaDTO.Descricao, 1, 500, "Descricao");
    ValidarTamanho(secretariaDTO.Logradouro, 1, 200, "Logradouro");

    if (secretariaDTO.IdSecretaria < 0)
    {
        throw new ArgumentException("IdSecretaria não pode ser negativo.");
    }
}
```

12 referências

```
private static void ValidarObrigatorio(string valor, string campo)
{
    if (string.IsNullOrEmpty(valor))
    {
        throw new ArgumentException($"Campo obrigatório não preenchido: {campo}.");
    }
}

private static void ValidarTamanho(string valor, int minimo, int maximo, string campo)
{
    if (valor.Length < minimo || valor.Length > maximo)
    {
        throw new ArgumentException($"Campo {campo} deve conter entre {minimo} e {maximo} caracteres.");
    }
}
```

1 referência

```
private static bool IsEmailValido(string email)
{
    try
    {
        var mailAddress = new MailAddress(email);
        return mailAddress.Address == email;
    }
    catch
    {
        return false;
    }
}
```

12 referências

```
private static string Sanitize(string? valor, bool digitsOnly = false)
{
    if (string.IsNullOrEmpty(valor))
    {
        return string.Empty;
    }

    var sanitized = valor.Trim();
    return digitsOnly
        ? Regex.Replace(sanitized, "[^0-9]", string.Empty)
        : sanitized;
}
```

E por fim foi adicionado a validação se o cnpj é verdadeiro.

```

private static bool IsCnpj(string cnpj)
{
    if (cnpj.Length != 14)
    {
        return false;
    }

    if (cnpj.All(caractere => caractere == cnpj[0]))
    {
        return false;
    }

    int[] pesoPrimeiroDigito = [5, 4, 3, 2, 9, 8, 7, 6, 5, 4, 3, 2];
    int[] pesoSegundoDigito = [6, 5, 4, 3, 2, 9, 8, 7, 6, 5, 4, 3, 2];

    var somaPrimeiroDigito = 0;
    for (var i = 0; i < 12; i++)
    {
        somaPrimeiroDigito += (cnpj[i] - '0') * pesoPrimeiroDigito[i];
    }

    var restoPrimeiroDigito = somaPrimeiroDigito % 11;
    var digito13 = restoPrimeiroDigito < 2 ? 0 : 11 - restoPrimeiroDigito;

    var somaSegundoDigito = 0;
    for (var i = 0; i < 13; i++)
    {
        var numero = i == 12 ? digito13 : cnpj[i] - '0';
        somaSegundoDigito += numero * pesoSegundoDigito[i];
    }

```

```

    var restoSegundoDigito = somaSegundoDigito % 11;
    var digito14 = restoSegundoDigito < 2 ? 0 : 11 - restoSegundoDigito;

    return cnpj[12] - '0' == digito13 && cnpj[13] - '0' == digito14;
}

```

Assinaturas